

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 21

СОЗДАНИЕ И УПРАВЛЕНИЕ МОДЕЛЯМИ ДАННЫХ. РАБОТА С АДМИНИСТРАТИВНОЙ ПАНЕЛЬЮ В DJANGO

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № _____ от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание и управление моделями данных. Работа с административной панелью в Django

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Использование моделей

Модель – единственный и окончательный источник информации о ваших данных. Он содержит основные поля и поведение хранимых вами данных. Обычно каждой модели соответствует одна таблица в базе данных.

Каждая модель – класс Python, который наследуется от `django.db.models.Model`.

Каждый атрибут модели представляет поле базы данных. Наряду со всем этим Django предоставляет API для доступа к автоматически сгенерированной базе данных.

После определения моделей Вам нужно сообщить Django, что вы хотите использовать эти модели. Вы сможете реализовать данный функционал, отредактировав файл настроек и изменив настройку, `INSTALLED_APPS` добавив имя модуля, который его содержит `models.py`

Например, если ваша модель приложения находится в модуле `myapp.models` (структура пакета, созданная для приложения скриптом), должна содержать: `manage.py startapp INSTALLED_APPS`

4.2 Подключение к базе данных

Для хранения данных в веб-приложении, как правило, применяются базы данных. И фреймворк Django уже по умолчанию предоставляет удобный функционал для работы с различными системами баз данных.

По умолчанию Django в качестве базы данных использует SQLite. Она очень проста в использовании и не требует запущенного сервера. Все файлы базы данных могут легко переноситься с одного компьютера на другой. Однако при

необходимости мы можем использовать в Django большинство распространенных СУБД.

Для работы с базами данных в проекте Django в файле settings.py определен параметр DATABASES, который по умолчанию выглядит следующим образом:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Переменная DATABASES содержит набор конфигураций подключений к базам данных в виде словаря. Ключи в этом словаре - названия подключений. То есть мы можем определить кучу подключений. Но как минимум одно подключение должно быть определено в переменной DATABASES - подключение с именем default, которое представляет подключение по умолчанию.

Конфигурация каждого подключения может состоять из ряда параметров. По умолчанию указываются только два параметра. Параметр ENGINE указывает на используемый движок для доступа к БД. В данном случае это встроенный пакет django.db.backends.sqlite3.

Второй параметр - NAME указывает на путь к базе данных. По умолчанию база данных называется db.sqlite3. Для установки пути используется каталог из переменной BASE_DIR, которая задана в начале файла:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

По умолчанию BASE_DIR указывает на каталог, в котором находится папка проекта. И после первого запуска проекта в указанном каталоге по умолчанию будет создан файл db.sqlite3, который, собственно, и будет использоваться в качестве базы данных (рисунок 1).

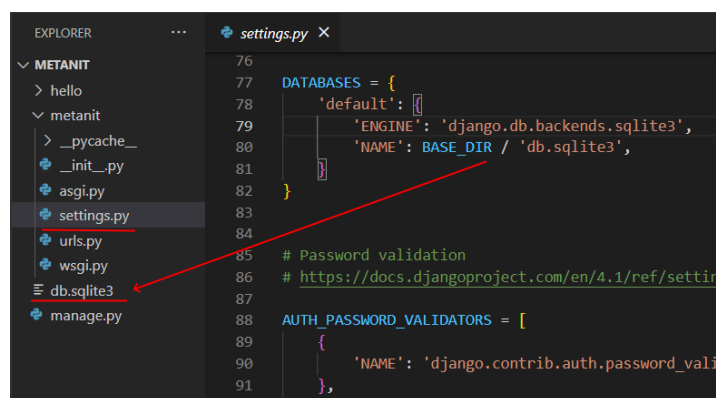


Рисунок 1 - Установка пути к базе данных в проекте Django

Чтобы использовать другие системы управления базами данных, необходимо будет установить соответствующий пакет.

Это встроенные движки, но также сообщество может предоставлять свои движки к другим СУБД. Например, компания Microsoft предоставляет пакет mssql-

django для работы с MS SQL Server (устанавливается командой `pip install mssql-django`)

Выше в настройках по умолчанию применялись только два параметра: ENGINE и NAME. Однако при необходимости можно указать и дополнительные параметры:

- ATOMIC_REQUESTS: при значении True оборачивает каждый запрос к бд в транзакцию. По умолчанию равно False;
- AUTOCOMMIT: при значении False отключает систему управления транзакциями Django. По умолчанию равно True;
- ENGINE: движок бд. Поддерживаются следующие встроенные движки:
 - django.db.backends.postgresql;
 - django.db.backends.mysql;
 - django.db.backends.sqlite3;
 - django.db.backends.oracle.
- HOST: хост базы данных;
- NAME: имя базы данных, для sqlite - путь к бд;
- CONN_MAX_AGE: время жизни подключения в секундах. Значение 0 закрывает подключение сразу после завершения каждого запроса к бд. А значение None указывает на постоянное подключение. По умолчанию равно 0;
- CONN_HEALTH_CHECKS: при значении True проверяет активность подключения, и если оно закрыто, заново устанавливает соединение. По умолчанию равно False;
- OPTIONS: дополнительный набор параметров, который может потребоваться при подключении к определенным базам данным;
- PASSWORD: пароль. По умолчанию равно "" (пустая строка);
- PORT: порт бд. По умолчанию равно "";
- TIME_ZONE: часовой пояс. По умолчанию равно None;
- USER: имя пользователя. По умолчанию равно "";
- TEST: набор настроек в виде словаря для теста подключения.

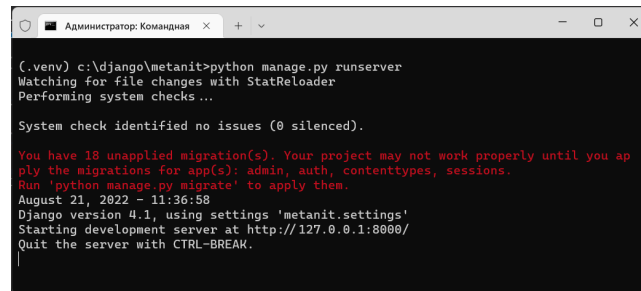
Для разных СУБД может потребоваться разный набор параметров. Например, подключение к БД Postgres в общем случае выглядит так:

```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'mydatabase',
        'USER': 'mydatabaseuser',
        'PASSWORD': 'mypassword',
        'HOST': '127.0.0.1',
        'PORT': '5432',
    }
}

```

При создании базы данных она еще пуста, не содержит никаких данных. Однако в консоли при запуске проекта мы можем увидеть некоторое сообщение, выделенное красным цветом (рисунок 2):



```

(.venv) c:\django\metanit>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you ap
ply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
August 21, 2022 - 11:36:58
Django version 4.1, using settings 'metanit.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2 - Применение миграций базы данных для встроенных приложений в Django

Дело в том, что для ряда приложений, подключенных в проект по умолчанию, требуется база данных для сохранения некоторых своих данных. Так, в файле settings.py в переменной INSTALLED_APPS по умолчанию указан ряд приложений

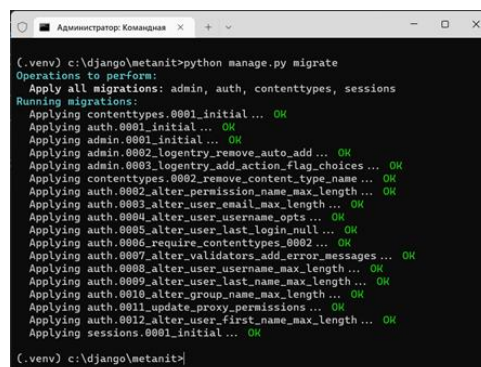
```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # остальные приложения .....
    # .....
]

```

Первые четыре приложения так или иначе используют базу данных. Однако, чтобы базы данных создала нужные для этих приложений таблицы, необходимо выполнить миграции - специальные скрипты, которые изменяют структуру базы данных. Для этого выполним в консоли команду (рисунок 3):

python manage.py migrate



```

(.venv) c:\django\metanit>python manage.py migrate
Operations to perform:
Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
Applying contenttypes.0001_initial ... OK
Applying auth.0001_initial ... OK
Applying admin.0001_initial ... OK
Applying admin.0002_logentry_remove_auto_add ... OK
Applying admin.0003_logentry_add_action_flag_choices ... OK
Applying contenttypes.0002_remove_content_type_name ... OK
Applying auth.0002_alter_permission_name_max_length ... OK
Applying auth.0003_alter_user_email_max_length ... OK
Applying auth.0004_alter_user_username_opts ... OK
Applying auth.0005_alter_user_last_login_null ... OK
Applying auth.0006_require_contenttypes_0002 ... OK
Applying auth.0007_alter_validators_add_error_messages ... OK
Applying auth.0008_alter_user_username_max_length ... OK
Applying auth.0009_alter_user_last_name_max_length ... OK
Applying auth.0010_alter_group_name_max_length ... OK
Applying auth.0011_update_proxy_permissions ... OK
Applying auth.0012_alter_user_first_name_max_length ... OK
Applying sessions.0001_initial ... OK
(.venv) c:\django\metanit>

```

Рисунок 3 - Миграции базы данных в Django

Если после этого мы откроем базу данных db.sqlite3, которая есть в проекте, в какой-нибудь специальной программе для просмотра БД SQLite, то мы увидим, что она содержит ряд таблиц (рисунок 4):

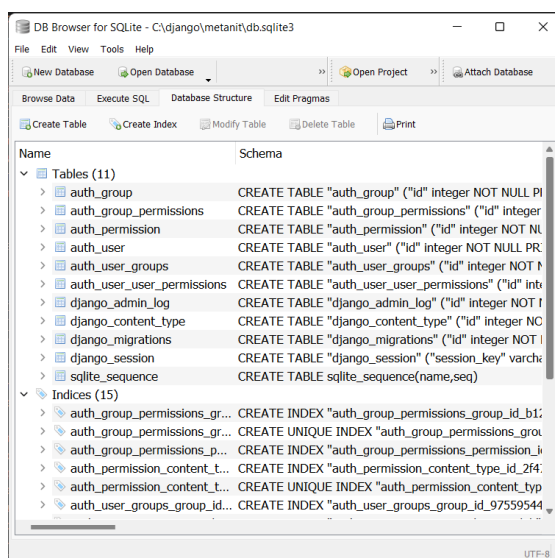


Рисунок 4 - Просмотр базы данных SQLite в Django

Для просмотра и работы с базами данных SQLite можно использовать бесплатную программу DB Browser for SQLite, которая доступна для всех основных операционных систем.

В частности, можно увидеть, что база данных содержит более десятка таблиц и индексов. Все они связаны с системой аутентификации и авторизации, администрирования и сессиями. И даже если мы непосредственно не будем использовать базу данных, но будем использовать системы аутентификации и авторизации, администрирования или сессии, то для нормальной работы нам все равно потребуется база данных и необходимо будет выполнить миграции.

4.3 Создание моделей

Модели в Django описывают структуру используемых данных. Используемые в программе данные хранятся в базах данных, и с помощью моделей как раз осуществляется взаимодействие с базой данных.

При создании приложения по умолчанию в его каталог добавляется файл models.py, который применяется для определения моделей. Модель представляет класс, унаследованный от django.db.models.Model.

Так, изменим файл models.py следующим образом (рисунок 5):

```
from django.db import models
class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

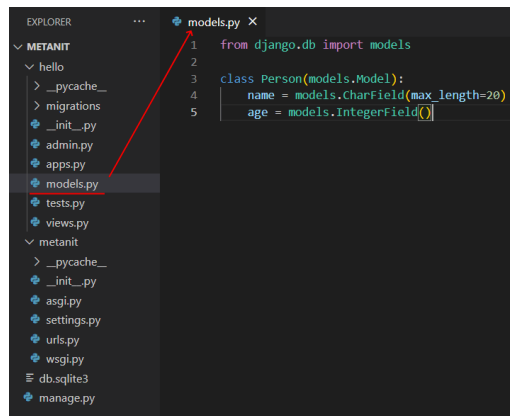


Рисунок 5 - Создание моделей в файле models.py в приложении на Django

Здесь определена простейшая модель, которая называется Person и которая представляет человека. В модели определены два поля. Поле name представляет тип CharField - текстовое поле, которое хранит последовательность символов. Оно будет хранить имя человека. Для CharField обязательно надо указать параметр max_length, который задает максимальную длину хранящейся строки. И поле age представляет тип IntegerField - числовое поле, которое хранит целые числа. Оно предназначено для хранения возраста человека.

Каждая модель сопоставляется с определенной таблицей в базе данных. Однако пока у нас нет в бд таблицы, которая хранит объекты модели Person. И в этом случае нам надо создать и выполнить миграцию. Миграция преобразует базу данных в соответствии с определением моделей.

Вначале необходимо создать миграцию с помощью команды (рисунок 6):
python manage.py makemigrations

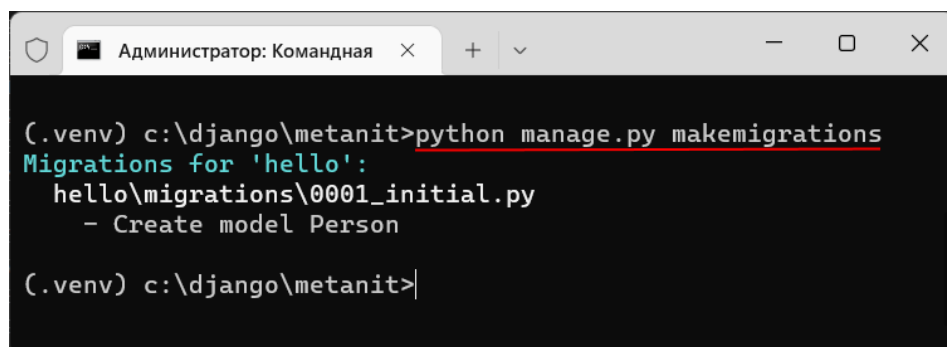


Рисунок 6 - Создание миграции моделей в Django

После этого в приложении в папке migrations мы обнаружим новый файл, который будет иметь примерно следующее содержимое:

```
from django.db import migrations, models
class Migration(migrations.Migration):
    initial = True
    dependencies = [
    ]
```

```

operations = [
    migrations.CreateModel(
        name='Person',
        fields=[
            ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
            ('name', models.CharField(max_length=20)),
            ('age', models.IntegerField()),
        ],
    ),
]

```

Это и есть миграция. Здесь можно заметить, что создается не два, а три поля - поле id, которое будет представлять первичный ключ, добавляется по умолчанию. Поэтому в принципе в самой модели нам не нужно явным образом определять какой-либо идентификатор (рисунок 7).

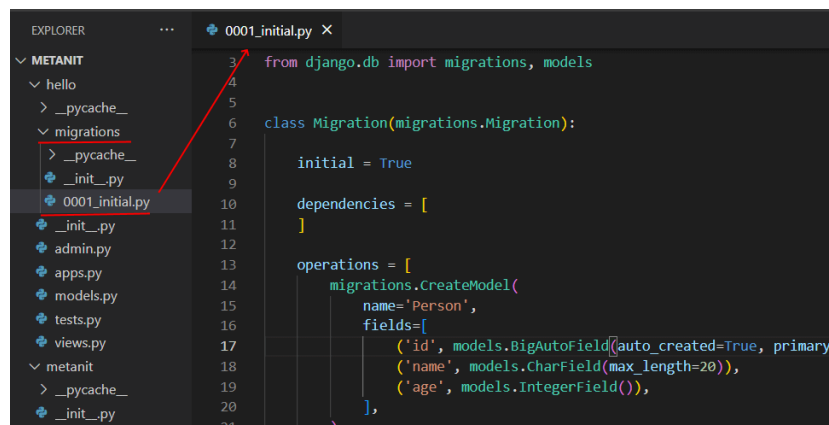


Рисунок 7 - Миграция моделей в Django

Теперь надо выполнить данную миграцию. Для этого выполняется команда (рисунок 8):

```
python manage.py migrate
```

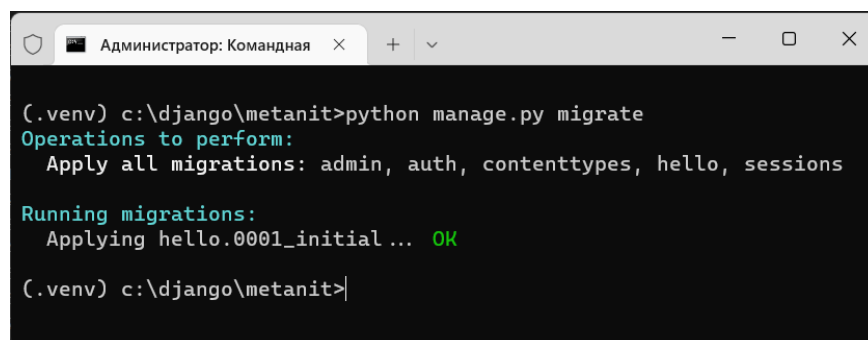


Рисунок 8 - Выполнение миграции базы данных в Django

После этого, если мы откроем базу данных db.sqlite3, которая есть в проекте, то мы увидим, что в нее добавлена таблица для хранения данных модели Person (рисунок 9):

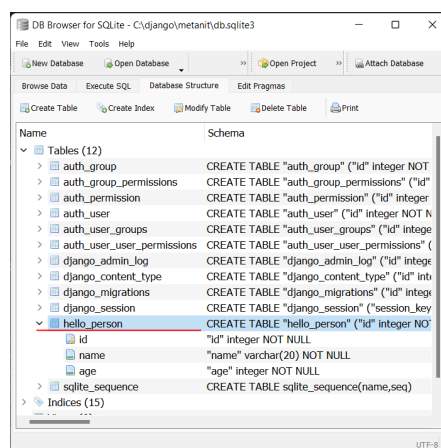


Рисунок 9 - Базы данных SQLite в Django

Стоит учитывать, что в процессе работы над проектом определение модели может меняться - к ней могут добавляться новые поля, либо могут удаляться уже существующие, либо могут добавляться новые модели, данные которых мы также захотим хранить в базе данных. И в этом случае опять надо создавать миграцию и применять ее к базе данных, чтобы база данных была синхронизирована с определением моделей.

4.4 Практическая часть

Для выполнения практической части практической работы, Вам необходимо развернуть Django-проект. Вспоминая, 20 лабораторную работу самостоятельно разверните проект и отобразите на сайте текст «Hello, world!»).

Подсказка по развертыванию проекта:

- `python -m venv venv;`
- `pip install django;`
- `django-admin startproject [имя проекта];`
- `./manage.py runserver` (или `python manage.py runserver`);
- `./manage.py migrate` (или `python manage.py migrate`);
- `django-admin startapp [имя приложения];`
- `INSTALLED_APPS =>` Добавить 'index';
- создать templates => Создать 'index.html' (В теге первого уровня добавить текст «Hello, world!»);
- `DIRS =>` Добавить 'templates';
- создать представление;
- добавить шаблон для URL-адресов (path).

Для добавления какого-либо функционала сайта, почти всегда применяют базу данных, но для того, чтобы мы не морочили себе голову с языком SQL, в Django все реализовано с помощью языка Python, прямо внутри кода. Базу данных

надо воспринимать как хранилище из таблиц.

Создать модель в БД = Создать таблицу в БД. Представим, что нам нужно реализовать работу с сотрудниками на сайте, т.е. нам необходимо создать список сотрудников. Для этого, зайдём в файл моделей приложения `index (models.py)`. И пропишем код (рисунок 10):

```
1 from django.db import models
2
3 class Worker(models.Model):
4     pass
```

Рисунок 10 - Пустая модель сотрудника (Worker)

Что здесь происходит? Мы создаем класс `Worker` (Сотрудник, работник), который наследуется от `models.Model` и также, ещё стоит заглушка `pass`, означающая то, что код ничего не будет делать.

Определим поля для нашей таблицы сотрудников, такие как имя, фамилия, зарплата (рисунок 11).

```
1 from django.db import models
2
3 class Worker(models.Model):
4     name = models.CharField(max_length=25, blank=False)
5     surname = models.CharField(max_length=50, blank=False)
6     salary = models.IntegerField(default=0)
```

Рисунок 11 - Заполненная модель сотрудника (Worker)

Что здесь происходит? Создаем переменные, которые будут иметь свой определенный тип данных.

Тип данных в программировании — классификация различных значений, которые могут использоваться в программах.

`CharField` — для работы со строковой информацией. `max_length` — максимальная длина строки не будет превышать n-символов и поле НЕ МОЖЕТ быть пустым.

`IntegerField` — для работы с числами. `default` — для установки значения по умолчанию (установим значение по умолчанию 0)

Готово! Но, Django не знает о нашей только что созданной модели — ничего!

Для того, чтобы модель появилась в нашем проекте, нужно провести миграции (рисунок 12).

```
PS D:\django-learning\hello\learning_django> python manage.py makemigrations
Migrations for 'index':
  index\migrations\0001_initial.py
  - Create model Worker
```

Рисунок 12 - Проведение миграций

И далее, необходимо выполнить проведенные миграции командой `python manage.py migrate` (рисунок 13).

```
PS D:\django-learning\hello\learning_django> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, index, sessions
Running migrations:
  Applying index.0001_initial... OK
```

Рисунок 13 - Выполнение миграций

Готово! Модель в БД создана!

Как создавать записи в нашей таблице сотрудников?

Необходимо зайти в админ-панель Django-проекта. Для этого, измените поисковую строку (рисунок 14):



Рисунок 14 - URL-адрес административной панели Django

К сожалению, или к счастью, без логина и пароля никто не разрешит нам войти в админ-панель нашего приложения. Для создания админа (или же — суперпользователя), перейдем в терминал и пропишем команду (рисунок 15):

```
PS D:\django-learning\hello\learning_django> python manage.py createsuperuser
Username: █
```

Рисунок 15 - Создание суперпользователя

Суперпользователь в Django — главный пользователь сайта с максимальными правами.

Придумываем имя для нашего админа, указываем почту (необязательно) и пароль.

При написании пароля, введенные нами символы в консоли отображаться НЕ БУДУТ!

Далее, если пароль будет не надежным, Django предупредит нас об этом, проигнорируем его, т.к. мы работаем на локальном сервере, и при необходимости мы сможем поменять пароль, если будет публиковать приложение на сервер, и отправляем в терминал символ `y` (yes) (рисунок 16).

```
PS D:\django-learning\hello\learning_django> python manage.py createsuperuser
Username: gazik
Email address:
Password:
Password (again):
The password is too similar to the username.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Рисунок 16 - Успешное создание суперпользователя

Войдем в админ-панель, используя только что созданные данные для входа (рисунок 17).

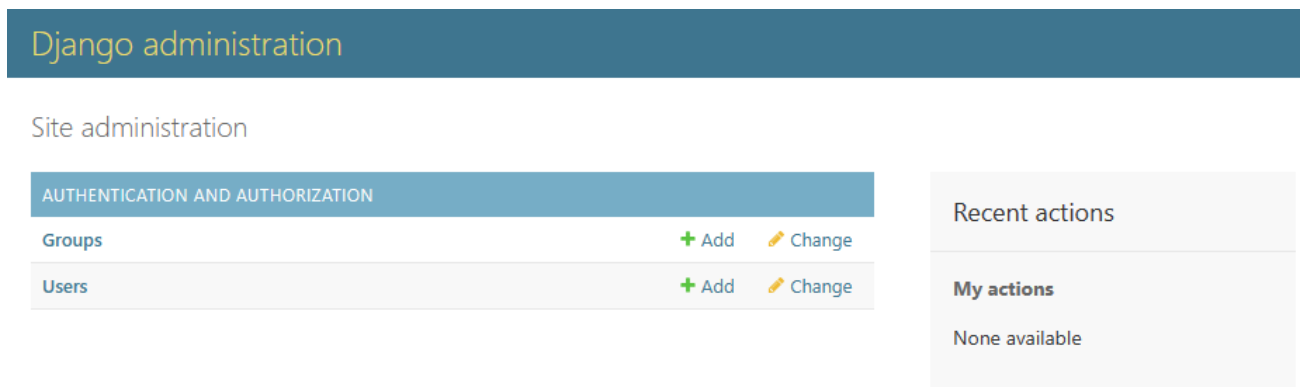


Рисунок 17 - Вид административной панели Django

Войдя в админ-панель, мы видим, автоматически созданную модель пользователей (User) (здесь будет отображаться список всех пользователей, которые регистрируются на сайте).

Зарегистрируем нашу модель в файле admin.py (рисунок 18).

```
1 from django.contrib import admin
2 from .models import Worker
3
4 admin.site.register(Worker)
```

Рисунок 18 - Регистрация модели в файле admin.py

Что здесь происходит? Мы импортируем из файла моделей модель сотрудника и регистрируем на сайте модель. Все очень просто (рисунок 19)!

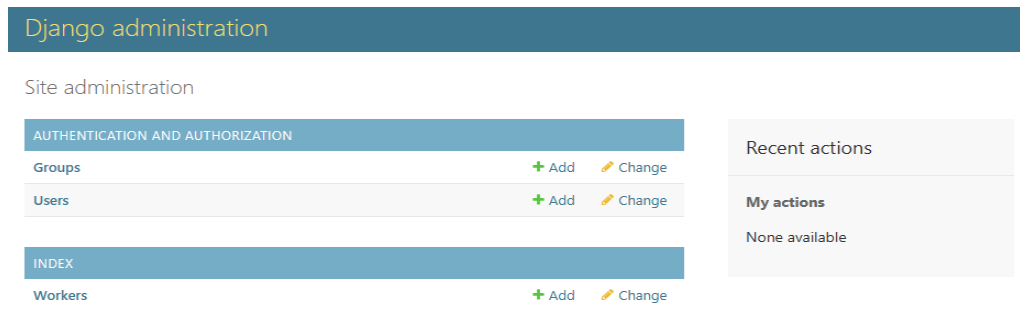


Рисунок 19 - Зарегистрированная модель Workers в админ-панели

Добавим сотрудника в БД. Нажмем на Add и потом, напишем имя, фамилию и зарплату (рисунок 20).

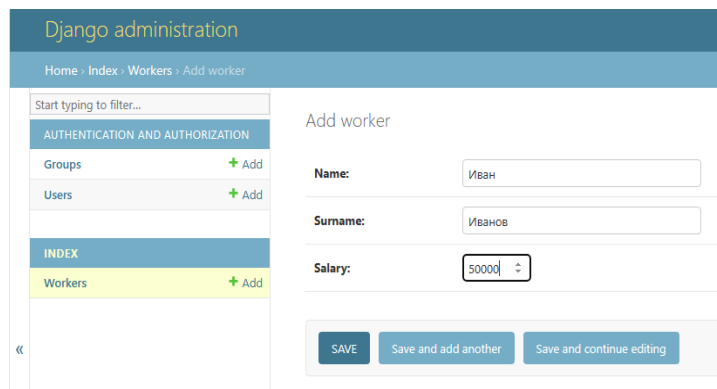


Рисунок 20 - Добавление сотрудника

Добавим ещё двух сотрудников, и что мы видим (рисунок 21)?

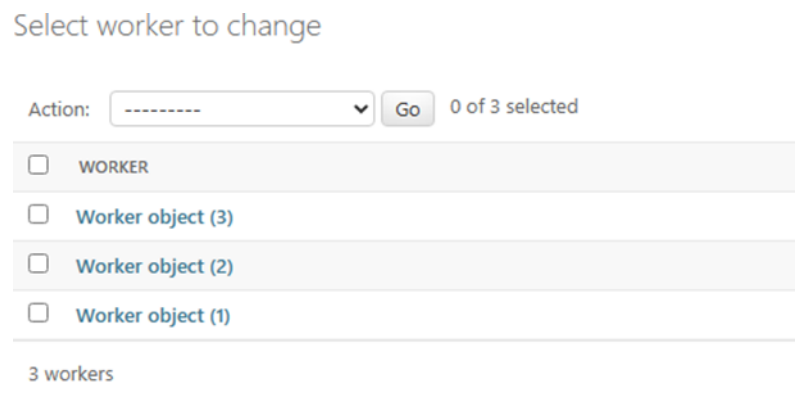


Рисунок 21 - Worker object (n)

Мы не можем понять, кто есть кто, исправим данный недочет. Зайдем в модели, и добавим код для магического метода `str`. Он отображает информацию об объекте класса для пользователей (рисунок 22).

```

1  from django.db import models
2
3  class Worker(models.Model):
4      name = models.CharField(max_length=25, blank=False)
5      surname = models.CharField(max_length=50, blank=False)
6      salary = models.IntegerField(default=0)
7
8  def __str__(self) -> str:
9      return f'{self.surname} {self.name}'
10

```

Рисунок 22 - Строковое представление для модели Worker

Теперь, наши сотрудники будут отображаться корректно. У них будет фамилия и имя (рисунок 23).

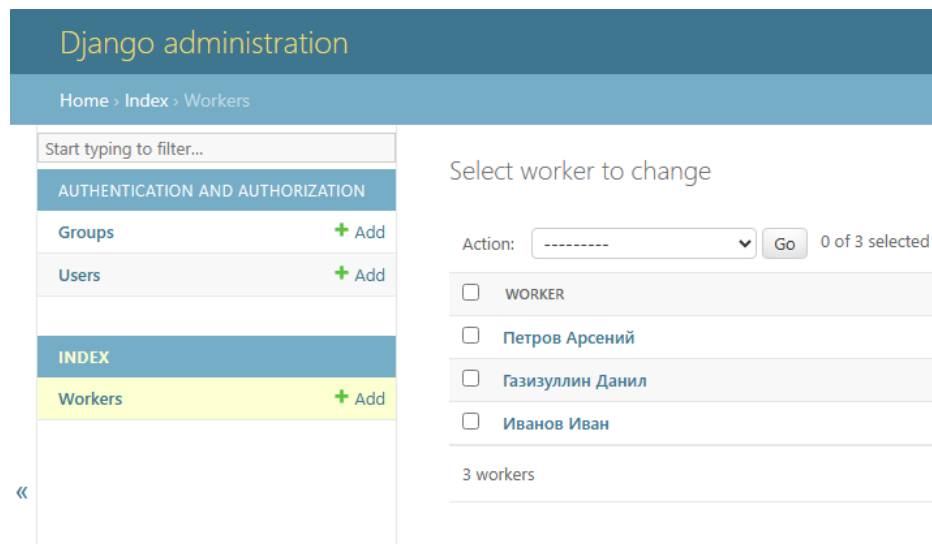


Рисунок 23 - Корректное отображение данных о сотрудниках в админ-панели

Ну согласитесь, так намного лучше!

Зайдем в файл представлений и пропишем код (рисунок 24):

```

1  from django.shortcuts import render
2  from .models import Worker
3
4  def index_page(request):
5      all_workers = Worker.objects.all()
6      print(all_workers)
7
8      return render(request, 'index.html')

```

Рисунок 24 - Вывод всех сотрудников модели Worker в консоли

Что здесь происходит? Мы создаем переменную, где будет храниться список всех сотрудников и выводим его в терминал (рисунок 25).

```
<QuerySet [<Worker: Иванов Иван>, <Worker: Газизуллин Данил>, <Worker: Петров Арсений>]>
[23/Jan/2024 17:46:38] "GET / HTTP/1.1" 200 233
```

Рисунок 25 - Вывод в консоли всех сотрудников модели Worker

А что, если нужно получить записи по некому фильтру! (рисунок 26)?

```
1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5     worker_salary = Worker.objects.filter(salary=70000)
6     print(worker_salary)
7     return render(request, 'index.html')
```

Рисунок 26 - Вывод данных по фильтру зарплаты в 70000

Мы получим результат в виде вывода данных в консоли с фильтром в виде зарплаты в 70000 (рисунок 27):

```
<QuerySet [<Worker: Газизуллин Данил>]>
[23/Jan/2024 17:51:27] "GET / HTTP/1.1" 200 233
```

Рисунок 27 - Отфильтрованный вывод данных в консоли

Красивый вывод списка всех сотрудников в консоли (имя, фамилия, зарплата) (рисунок 28).

```
1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5
6     all_workers = Worker.objects.all()
7
8     for worker in all_workers:
9         print(worker.id, worker.name, worker.surname, worker.salary)
10
11
12     return render(request, 'index.html')
```

Рисунок 28 - Вывод списка всех сотрудников в консоли через цикл for

Если посмотреть в консоль, то мы увидим следующий вывод (рисунок 29):

```
1 Иван Иванов 50000
2 Данил Газизуллин 70000
3 Арсений Петров 105000
[23/Jan/2024 19:53:46] "GET / HTTP/1.1" 200 233
```

Рисунок 29 - Вывод данных в консоли через цикл for

Как добавить новую запись? По простым принципам ООП. Советую изучить работу с ООП и знать её хотя бы на базовом уровне (рисунок 30).

```

1  from django.shortcuts import render
2  from .models import Worker
3
4  def index_page(request):
5
6      all_workers = Worker.objects.all()
7
8      leila_gatufovna = Worker(name='Лейла', surname='Галимова', salary='200000')
9      leila_gatufovna.save()
10
11     return render(request, 'index.html')

```

Рисунок 30 - Добавление нового сотрудника

Как изменить запись? Все очень просто (рисунок 31):

```

1  from django.shortcuts import render
2  from .models import Worker
3
4  def index_page(request):
5
6      all_workers = Worker.objects.all()
7
8      changed_worker = Worker.objects.get(id=1)
9      changed_worker.surname = 'Дубров'
10     changed_worker.save()
11
12     return render(request, 'index.html')

```

Рисунок 31 - Редактирование сотрудника

Если посмотреть таблицу сотрудников, то можно заметить, что «Иван Иванов» стал «Иван Дубров» (рисунок 32).

Select worker to change

Action: 0 of 4 selected

☐	WORKER
☐	Галимова Лейла
☐	Петров Арсений
☐	Газизуллин Данил
☐	Дубров Иван

4 workers

Рисунок 32 - Измененные данные сотрудника (фамилия)

Много кода, не так ли? Воспользуемся специальным методом update (рисунок 33).

```

1  from django.shortcuts import render
2  from .models import Worker
3
4  def index_page(request):
5
6      Worker.objects.filter(id=1).update(name='Дмитрий')
7
8      return render(request, 'index.html')

```

Рисунок 33 - Использование метода update для редактирования данных

И мы также увидим изменение в данных сотрудника (рисунок 34):

Select worker to change

Action: 0 of 4 selected

☐	WORKER
☐	Галимова Лейла
☐	Петров Арсений
☐	Газизуллин Данил
☐	Дубров Дмитрий

4 workers

Рисунок 34 - Измененные данные сотрудника (имя)

Как удалить запись в таблице? Не прижился, ты Иван (Дмитрий) Иванов (Дубров), пора бы тебя и удалить. Для этого воспользуемся фильтром по id и методом delete (рисунок 35).

```

1  from django.shortcuts import render
2  from .models import Worker
3
4  def index_page(request):
5
6      Worker.objects.filter(id=1).delete()
7
8      return render(request, 'index.html')

```

Select worker to change

Action: 0 of 3 selected

☐	WORKER
☐	Галимова Лейла
☐	Петров Арсений
☐	Газизуллин Данил

3 workers

Рисунок 35 - Использование фильтра и метода delete для удаления данных из таблицы Worker

4.5 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте blog_site и приложении posts создайте модель Post, предназначенную для хранения публикаций блога. Модель должна содержать следующие поля: заголовок публикации, текст публикации, дата создания и автор. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле admin.py.

Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти публикаций блога.

Решение задачи:

1. Создание модели (models.py)

```
from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=255)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    author = models.CharField(max_length=100)
    def __str__(self):
        return self.title
```

2. Выполнение миграций

```
python manage.py makemigrations
python manage.py migrate
```

3. Регистрация модели в админке (admin.py)

```
from django.contrib import admin
from .models import Post
admin.site.register(Post)
```

4. Создание администратора

```
python manage.py createsuperuser
```

5. Запуск сервера

```
python manage.py runserver
```

6. Проверка работы

Откройте в браузере:

<http://127.0.0.1:8000/admin>

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте `library_site` и приложении `books` создайте модель `Book`, предназначенную для хранения информации о книгах. Модель должна содержать следующие поля: название книги, автор, год издания, количество страниц и краткое описание. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти записей о книгах.

2 В ранее созданном проекте `student_portal` и приложении `students` создайте модель `Student`, предназначенную для хранения информации о студентах. Модель должна содержать следующие поля: фамилия, имя, возраст, группа и дата

поступления. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о студентах.

3 В ранее созданном проекте `movie_portal` и приложении `films` создайте модель `Film`, предназначенную для хранения информации о фильмах. Модель должна содержать следующие поля: название фильма, режиссёр, год выпуска, жанр и рейтинг. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о фильмах.

4 В ранее созданном проекте `music_library` и приложении `albums` создайте модель `Album`, предназначенную для хранения информации о музыкальных альбомах. Модель должна содержать следующие поля: название альбома, исполнитель, год выпуска, количество песен и жанр. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о музыкальных альбомах.

5 В ранее созданном проекте `online_shop` и приложении `products` создайте модель `Product`, предназначенную для хранения информации о товарах интернет-магазина. Модель должна содержать следующие поля: название товара, цена, количество на складе, описание товара и дата добавления. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о товарах.

6 В ранее созданном проекте `news_portal` и приложении `news` создайте модель `News`, предназначенную для хранения информации о новостях. Модель должна содержать следующие поля: заголовок новости, текст новости, дата публикации, автор и категория. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти новостей.

7 В ранее созданном проекте `tourism_portal` и приложении `tours` создайте модель `Tour`, предназначенную для хранения информации о туристических маршрутах. Модель должна содержать следующие поля: название маршрута, страна, длительность тура, стоимость и описание маршрута. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора

системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти туристических маршрутов.

8 В ранее созданном проекте `sport_club_site` и приложении `teams` создайте модель `Team`, предназначенную для хранения информации о спортивных командах. Модель должна содержать следующие поля: название команды, вид спорта, город, год основания и количество игроков. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти спортивных команд.

9 В ранее созданном проекте `school_portal` и приложении `teachers` создайте модель `Teacher`, предназначенную для хранения информации о преподавателях. Модель должна содержать следующие поля: фамилия, имя, предмет преподавания, стаж работы и номер кабинета. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти преподавателей.

10 В ранее созданном проекте `hospital_system` и приложении `patients` создайте модель `Patient`, предназначенную для хранения информации о пациентах. Модель должна содержать следующие поля: фамилия, имя, дата рождения, диагноз и дата поступления. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти записей о пациентах.

11 В ранее созданном проекте `education_platform` и приложении `courses` создайте модель `Course`, предназначенную для хранения информации об учебных курсах. Модель должна содержать следующие поля: название курса, преподаватель, длительность курса, стоимость обучения и описание курса. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти учебных курсов.

12 В ранее созданном проекте `restaurant_site` и приложении `menu` создайте модель `Dish`, предназначенную для хранения информации о блюдах ресторана. Модель должна содержать следующие поля: название блюда, категория блюда, цена, описание и калорийность. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу <http://127.0.0.1:8000/admin>. Добавьте через административную панель не менее пяти блюд.

13 В ранее созданном проекте `car_catalog` и приложении `cars` создайте модель `Car`, предназначенную для хранения информации об автомобилях. Модель должна содержать следующие поля: марка автомобиля, модель автомобиля, год выпуска, объём двигателя и цена. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти автомобилей.

14 В ранее созданном проекте `portfolio_site` и приложении `main` создайте модель `Project`, предназначенную для хранения информации о проектах портфолио. Модель должна содержать следующие поля: название проекта, описание, дата создания и ссылка на проект. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о проектах.

15 В ранее созданном проекте `home_site` и приложении `home` создайте модель `Page`, предназначенную для хранения информации о страницах сайта. Модель должна содержать следующие поля: название страницы, текст страницы, дата создания и статус публикации. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти записей о страницах сайта.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

- 6.1 Тема лабораторной работы
- 6.2 Цель лабораторной работы
- 6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

- 7.1 Что такое модель в Django в языке программирования Python?
- 7.2 Для чего нужны миграции в языке программирования Python?
- 7.3 Какая команда создаёт миграции в языке программирования Python?
- 7.4 Для чего используется административная панель Django в языке программирования Python?
- 7.5 Как создать администратора в Django в языке программирования Python?
- 7.6 Для чего используется файл `admin.py` в языке программирования Python?
- 7.7 По какому адресу открывается административная панель Django в языке программирования Python?

Список используемых источников

- 1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
- 2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
- 3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.